CS543 • WEB SERVICES

ASSIGNMENT 6 OF 12

Honest Responses: Status Codes, Errors & Validation

Group ID: 17

TEAM MEMBERS

Student Name	Roll Number
Anurag Tiwari	20251651028
Saurabh Singh	20251651089
Mohd Rashid	20251651057
Niraj Kumar Gupta	20251651066

Part B — One Error Shape

Problem Details, validation errors, and safe exception handling

B1. Short theory

A shared error format makes it easier for React to show clear messages. The response uses type, title, status, and detail. For validation, add errors[] with the name of each invalid field and a reason. Use Content-Type: application/problem+json and never expose stack traces or database messages.

B1.5 Error type catalogue

Error type	Status	When it happens
unsupported-algorithm	422	Algorithm is not supported.
invalid-input	422	Input values or array are invalid.
visualization-not-found	404	Requested run is missing or inaccessible.
illegal-transition	409	Action is not allowed in the current state.
idempotency-key-conflict	409	Same idempotency key used with a different body.
rate-limit-exceeded	429	Too many requests.
service-unavailable	503	Temporary service outage.
internal-error	500	Unexpected server-side problem.

B2. Problem Details model

```
public record ApiProblem(
String type,
String title,
int status,
String detail,
List<FieldError> errors) {}

public record FieldError(String field, String reason) {}
```

B3. Central exception handler

```
@RestControllerAdvice
public class ApiExceptionHandler {
@ExceptionHandler(MethodArgumentNotValidException.class)
ResponseEntity<ApiProblem> validation(
MethodArgumentNotValidException ex) {
List<FieldError> errors = ex.getBindingResult()
.getFieldErrors().stream()
.map(e -> new FieldError(
e.getField(), e.getDefaultMessage()))
.toList();

ApiProblem body = new ApiProblem(
"https://algoviz.example/problems/invalid-input",
"Invalid visualization input", 422,
"Correct the fields listed in errors.", errors);

return ResponseEntity.unprocessableEntity()
.contentType(MediaType.valueOf(
"application/problem+json"))
.body(body);
}

@ExceptionHandler(Exception.class)
ResponseEntity<ApiProblem> unexpected(Exception ex) {
// Log ex with a request/correlation ID on the server.
ApiProblem body = new ApiProblem(
"https://algoviz.example/problems/internal-error",
"Internal server error", 500,
"The request could not be completed.", List.of());
return ResponseEntity.status(500)
.contentType(MediaType.valueOf(
"application/problem+json"))
.body(body);
}
}
```

B4. Example 422 response

```
{
"type": "https://algoviz.example/problems/invalid-input",
"title": "Invalid visualization input",
"status": 422,
"detail": "Correct the fields listed in errors.",
"errors": [
{"field": "algorithm", "reason": "Unsupported algorithm."},
{"field": "input", "reason": "Input must not be empty."}
]
}
```

Notes / implementation caution

Use the same error contract for known failures across endpoints. In production, avoid a broad exception handler that accidentally hides useful distinctions: map known exceptions to suitable 4xx statuses and keep unknown exceptions as 500.